

PITCHPRESENCE

V1 Product Requirements Document & Build Flow

Attendance • Training Sessions • Monthly Dues • Paystack • Management

Implementation target: Codex-assisted build

Architecture: Next.js + Node.js/TypeScript + PostgreSQL/Prisma

Version: 1.0 • October 2026

1. Product Overview

PitchPresence is a mobile-first Progressive Web App for football teams. V1 focuses on two operational problems: (1) fast, trustworthy training attendance for players who arrive at different times, and (2) simple monthly-dues record keeping with optional in-app Paystack payment. The product should minimize coach administration and make player actions take only a few seconds.

2. V1 Goals

- Allow a coach or manager to open one attendance window for a training session and leave it open while players arrive at different times.
- Make normal player check-in camera scan → validation → success, with no attendance form.
- Reduce ghost attendance using short-lived rotating QR tokens tied to an open session.
- Capture the coach/manager device location once when attendance is opened, without continuously tracking the device.
- Record a server-side arrival timestamp and prevent duplicate attendance per player/session.
- Provide a manual attendance fallback with an audit trail.
- Track monthly dues independently for every active player and preserve month-by-month history.
- Allow players to pay dues with Paystack while still supporting payments made outside the app.
- Allow coach/management to manually mark external payments as paid.

3. V1 Non-Goals

- No automatic lateness classification in V1; arrival time is stored so lateness rules can be added later.
- No continuous player or coach GPS tracking.
- No requirement that dues have a fixed platform-defined amount.
- No automatic overdue penalties, subscriptions, proration, or attendance restriction based on dues status.
- No claim that rotating QR alone proves physical presence; it is an anti-abuse layer rather than absolute proof.

4. Users and Roles

Role	Capabilities
Player	Register/verify account; use remembered device; scan attendance QR; view attendance history; view current and historical dues; initiate Paystack payment.
Coach / Manager	Start/end attendance; establish session location; display rotating QR; view live attendance; manually mark attendance; view dues; manually mark external dues payments.

V1 can use one management permission set for Coach/Manager. More granular roles can be introduced later.

5. Proposed Technical Architecture

The original Go token service is replaced with Node.js/TypeScript. A practical V1 monorepo:

```
pitchpresence/  
  apps/  
    web/    Next.js 16, React, TypeScript, Tailwind, PWA  
    api/    Node.js + TypeScript API (Express or Fastify)
```

packages/
 shared/ shared types, schemas, constants
prisma/
 schema.prisma
package.json
.env.example

Layer	Responsibility
Next.js Web	Player and management UI, PWA shell, authenticated pages, QR display/scan landing experience.
Node.js API	Authentication orchestration, session lifecycle, QR token issuance/validation, attendance writes, dues/payment endpoints, Paystack webhook/verification.
PostgreSQL + Prisma	Persistent source of truth for users, sessions, attendance, monthly dues and payments.
Resend	Email OTP delivery for onboarding/verification.
Paystack	Optional in-app payment channel; successful transactions are verified server-side before a dues record becomes paid.

6. Authentication & Onboarding Flow

1. Player opens a team invite/registration link.
2. Player enters name, email and a 4-digit PIN.
3. Backend hashes the PIN, creates the player in an unverified state, creates a six-digit email OTP with expiry, and sends it through Resend.
4. Player enters OTP. Backend validates it, marks the account verified, and establishes a secure HTTP-only remembered-device session/cookie.
5. Future attendance scans from that device use the fast path and should not require another form.
6. If a scan is opened from an unrecognized device, the player authenticates with email + PIN. After successful authentication the device is remembered and the pending attendance attempt continues.

7. Training Session Lifecycle

7.1 Starting Attendance

7. Coach/Manager opens the management dashboard and chooses Start Attendance.
8. The web app requests the manager device's current geolocation once.
9. If a usable location is obtained, the backend creates/opens the training session with start time, captured latitude/longitude, location accuracy and status OPEN.
10. The attendance screen begins requesting/displaying short-lived signed QR tokens for that session.
11. The session remains OPEN independently of individual QR token expiration.

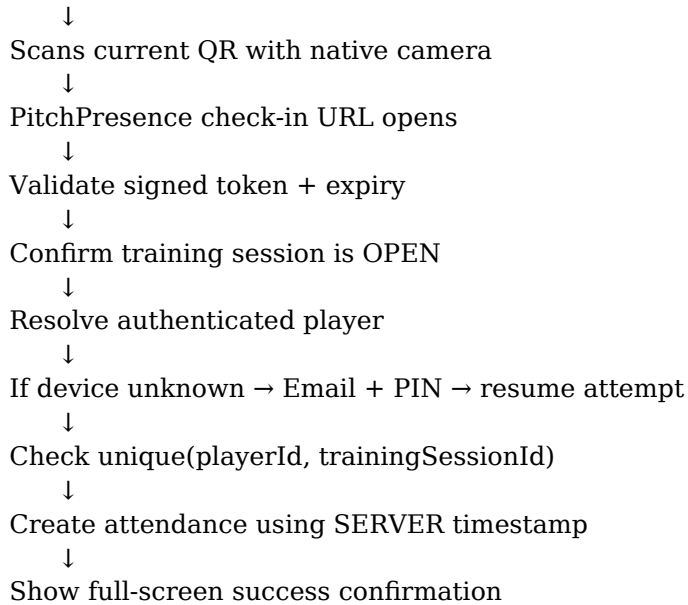
7.2 Rotating QR

Each displayed QR represents a cryptographically signed, short-lived token, approximately 15 seconds in V1. The token identifies the training session and contains issuance/expiry data and a nonce/token identifier. Sensitive location data should remain server-side rather than being embedded in the QR.

Validation	Required result
Signature	Valid
Token expiry	Not expired
Session	Exists and OPEN
Player	Authenticated and active
Duplicate check	No existing attendance for player + session

7.3 Player Check-In

PLAYER ARRIVES



Players can therefore arrive at 06:50, 07:05, 07:30 or later while the same training session remains open. Each receives the currently displayed QR and gets an independent arrival timestamp.

7.4 Manual Attendance

Coach/Manager can mark an active player present when the player cannot scan (dead phone, no internet, forgotten phone, camera problem, etc.). Manual attendance must store the timestamp, method=MANUAL and the manager user who performed the action. QR check-ins use method=QR.

7.5 Ending Attendance

Coach/Manager selects End Attendance. The backend changes the session to CLOSED and records closedAt. All subsequent scans must fail because session status is authoritative even if a QR token has a few seconds of cryptographic lifetime remaining.

7.6 Absence

V1 derives absence from the roster: after a session is closed, any active player with no attendance record for that session is absent. This avoids creating unnecessary ABSENT rows while preserving a complete report.

8. Anti-Ghosting & Attendance Integrity

Layer	Purpose
Manager location at session start	Records where the training attendance window was opened.
~15-second rotating signed QR	Makes old screenshots quickly unusable.
Open-session validation	A valid-looking token cannot check in after attendance closes.
Authenticated player/device	QR possession alone does not identify the attendee.
Unique DB constraint	One attendance row per player per training session.
Server timestamp	Player cannot choose/manipulate arrival time.
Audit method	Distinguishes QR check-in from management manual override.

Known V1 limitation: manager-device location does not independently prove the player's location. Real-time QR sharing can still be attempted. Player-location verification can be added later if abuse justifies the extra permission/friction.

9. Attendance UX Requirements

Screen	Minimum content/actions
Management Dashboard	Start Attendance; active session indicator; live checked-in count; dues summary.
Open Attendance	Session status; large rotating QR; refresh/expiry indicator; checked-in count; View Attendance; End Attendance.
Live Attendance	Player name, arrival time, method; unchecked roster; manual Mark Present action.
Player Success	Large success state, training name/date and recorded arrival time.
Expired/Invalid QR	Clear retry instruction: scan the QR currently displayed by management.
Closed Session	Attendance closed message; no attendance write.
Player Attendance History	Past training dates, present/absent result and arrival time where present.

10. Monthly Dues Requirements

Dues are monthly status records, not a subscription engine. PitchPresence is primarily the record keeper. Players may continue paying externally using account details shared outside the app, or optionally pay through Paystack.

10.1 Monthly State

Every active player must have a logical dues status for each calendar month. A new month begins NOT_PAID. The previous month's record is never reset or overwritten; historical paid/non-paid decisions remain available.

September 2026 PAID

October 2026 PAID

November 2026 NOT_PAID

10.2 Paystack Payment Flow

12. Player logs in and opens Dues.
13. Player chooses the current month and selects Pay with Paystack.
14. The app initializes a Paystack transaction with the player/month context and amount entered/selected according to the final UI rule.
15. Player completes Paystack checkout.
16. Backend must verify the transaction server-side and/or process the Paystack webhook. Browser success alone must not mark dues as paid.
17. On verified success, create the payment record and set that player's monthly dues status to PAID.
18. The result immediately appears in both the player's dues history and management dues view.

10.3 External Payment Flow

19. Player pays outside PitchPresence (for example, using account details shared in the team WhatsApp group).
20. Coach/Manager opens the player's current-month dues record.
21. Coach/Manager selects Mark as Paid.
22. Backend updates that month's status to PAID and records payment source/method as MANUAL/EXTERNAL, paidAt and markedBy.

23. The player and management immediately see the same updated monthly record.

10.4 Dues History

Management should be able to inspect a month-by-month matrix or filtered list showing paid/not-paid status for all active players. Players see only their own monthly history. Paystack references should be retained for Paystack transactions; manual records retain the manager who confirmed them.

11. Core Data Model (Proposed)

Model	Important fields / rules
User	id UUID; role PLAYER MANAGER; name; email unique; pinHash; isVerified; active; createdAt.
VerificationToken	id; email/userId; otpHash; expiresAt; consumedAt.
DeviceSession	id; userId; tokenHash/session identifier; expiresAt; lastUsedAt; revokedAt.
TrainingSession	id; date; startedAt; closedAt nullable; status OPEN CLOSED; startedBy; latitude; longitude; locationAccuracy.
Attendance	id; playerId; trainingSessionId; checkedInAt; method QR MANUAL; recordedBy nullable; createdAt. UNIQUE(playerId, trainingSessionId).
MonthlyDues	id; playerId; month (normalized month key); status PAID NOT_PAID; paidAt nullable; paymentMethod nullable; markedBy nullable. UNIQUE(playerId, month).
Payment	id; playerId; monthlyDuesId; amount; provider PAYSTACK EXTERNAL; providerReference nullable; status PENDING SUCCESS FAILED; paidAt; metadata; createdAt.

Implementation note: MonthlyDues can be created eagerly at month start or lazily on first access. Either approach must produce the same user-visible rule: every active player begins each new month as NOT_PAID while old months remain immutable historical records.

12. API Surface (Suggested)

Method	Route	Purpose
POST	/auth/register	Create player and send verification OTP.
POST	/auth/verify-email	Verify OTP and establish remembered device.
POST	/auth/device-login	Email + PIN fallback for unknown device.
POST	/training-sessions	Manager creates/opens session with captured location.
POST	/training-sessions/:id/qr-token	Issue current short-lived signed QR token.
POST	/attendance/check-in	Validate QR token and create QR attendance.
POST	/training-sessions/:id/attendance/manual	Manager manually marks a player present.
POST	/training-sessions/:id/close	Close attendance immediately.
GET	/training-sessions/:id/attendance	Live roster/attendance view.
GET	/me/attendance	Player attendance history.
GET	/me/dues	Player monthly dues history/current status.
POST	/payments/paystack/initialize	Initialize Paystack payment for a dues record.
POST	/payments/paystack/webhook	Receive and verify Paystack events.
POST	/dues/:id/mark-paid	Manager confirms an external payment.
GET	/management/dues	Management dues matrix/list.

13. Key Backend Rules

- Never trust client-supplied arrival timestamps; use server time.
- Never mark Paystack dues paid from a frontend callback alone; verify server-side.
- Attendance writes must be idempotent and protected by a database unique constraint.
- A QR token is useful only while both the token and its training session are valid.
- Closing a session takes precedence over remaining QR token lifetime.
- Only Coach/Manager can start/close sessions, manually mark attendance, or manually mark dues paid.
- Manual actions must record who performed them.
- PINs, OTPs and persistent authentication tokens must never be stored in plaintext.
- Paystack webhook processing must be idempotent so duplicate webhook delivery cannot duplicate payments.
- Historical monthly dues records must not be destroyed when a new month begins.

14. Important Edge Cases

Scenario	Expected behavior
Player scans expired QR	Reject; instruct player to scan currently displayed QR.
Player scans after session closed	Reject even if token signature/expiry is otherwise valid.
Player scans twice	Return existing/success state; do not create duplicate attendance.
Unknown device scans valid QR	Preserve pending token/context through login where safely possible, then complete check-in if still valid/open; otherwise request a fresh scan.
Manager location denied/unavailable	Do not silently invent location. V1 should show a clear retry/permission state; product may later define an explicit location-less override.
Player has no phone	Manager manual attendance.
Paystack callback says success but verification fails	Keep dues NOT_PAID/payment pending or failed; do not trust callback.
Paystack webhook delivered twice	Process once using provider reference/idempotency.
Manager marks already-paid month	Prevent duplicate state change or require an explicit audited correction workflow.
New calendar month	Current month appears NOT_PAID; all previous records remain unchanged.
Player joins mid-month	Create/logically expose the current month's NOT_PAID record; no proration logic in V1.

15. Acceptance Criteria

- A verified player on a remembered device can scan a current QR and receive a successful attendance result without entering a form.
- Two players arriving 30+ minutes apart can check into the same still-open training session using the QR displayed at their respective arrival times.
- A QR token older than the configured lifetime is rejected.
- Closing attendance prevents all new check-ins immediately.
- A player cannot have more than one attendance record for one training session.
- Management can manually mark a player present and the record identifies it as manual and identifies the manager.
- Attendance reports show arrival timestamps for present players and derive absent players from the active roster.
- A player's new monthly dues state is NOT_PAID without modifying prior-month records.
- A verified Paystack payment changes only the intended player's intended monthly dues record to PAID and stores its reference.
- Management can mark an externally paid player's month PAID with an audit trail.

- Player and management views reflect the same persisted dues state.

16. Suggested Codex Build Order

Phase 1 — Foundation

Create monorepo, Next.js PWA, Node.js TypeScript API, Prisma/PostgreSQL, shared types, environment validation and basic CI/lint/test commands.

Phase 2 — Identity

Implement User, VerificationToken and DeviceSession; registration, Resend OTP, email verification, PIN hashing, secure cookie/session and role authorization.

Phase 3 — Training Sessions

Implement TrainingSession model, manager start flow, one-time geolocation capture, OPEN/CLOSED lifecycle and management session screen.

Phase 4 — Rotating QR

Implement signed short-lived token issuance, QR display refresh, token validation and session-status validation.

Phase 5 — Attendance

Implement fast-path check-in, unknown-device fallback, unique DB constraint, server timestamps, live attendance list, manual attendance and history.

Phase 6 — Monthly Dues

Implement MonthlyDues, current-month resolution, historical views, management matrix and manual Mark as Paid with audit fields.

Phase 7 — Paystack

Implement transaction initialization, secure verification/webhook handling, idempotency, Payment records and automatic MonthlyDues update.

Phase 8 — Hardening

Add authorization tests, QR expiry/replay tests, duplicate attendance tests, webhook idempotency tests, audit logging, rate limiting, error states and deployment configuration.

17. Suggested Repository Commands

```
npm run dev          # run web + api
npm run dev:web      # Next.js only
npm run dev:api      # Node API only
npm run build
npm run lint
npm run test
npm run db:migrate
npm run db:studio
```

18. Environment Variables (Names Only)

DATABASE_URL
APP_URL
API_URL
SESSION_SECRET
QR_SIGNING_SECRET
RESEND_API_KEY
EMAIL_FROM
PAYSTACK_SECRET_KEY
PAYSTACK_PUBLIC_KEY
PAYSTACK_WEBHOOK_SECRET (if used by chosen verification approach)

Secrets must be server-only. Do not expose signing secrets, Resend keys or Paystack secret keys to browser code.

19. Open Product Decisions Before/While Building

- Training creation: manually create each session versus recurring schedule/auto-prepared sessions.
- Exact QR lifetime/refresh cadence (15 seconds is the current V1 target).
- Minimum acceptable manager geolocation accuracy and whether management gets an explicit override if location cannot be captured.
- Exact UI rule for Paystack amount entry because V1 has no platform-defined fixed dues amount.
- Whether a manager can reverse/correct an incorrectly marked dues payment, and the required audit trail.
- Whether multiple teams/organizations are required in V1. The current working scope assumes one team context unless multi-team support is explicitly added.

20. End-to-End V1 Flow

ONBOARDING

Invite → Register → Email OTP → Verified → Device remembered

ATTENDANCE

Manager → Start Attendance → Capture manager location → Session OPEN
→ Rotating ~15s QR
→ Player arrives at any time → Scan current QR
→ Validate token + OPEN session + authenticated player + uniqueness
→ Server records arrival → Success
→ Manager may manually add exceptions
→ End Attendance → Session CLOSED → No further check-ins
→ Attendance history/report

DUES

New month → Player's month = NOT_PAID
→ Option A: Paystack → server verification/webhook → Payment SUCCESS → month PAID
→ Option B: External transfer/cash → Manager Mark as Paid → month PAID
→ Player + Management see same state
→ Next month begins NOT_PAID while previous months remain in history

21. Definition of V1 Complete

V1 is complete when a team can onboard players, run a real training session with staggered arrivals using rotating QR check-in, handle manual attendance exceptions, close and review attendance, maintain month-by-month dues status, accept verified Paystack dues payments, manually confirm external dues payments, and view consistent historical records from both player and management interfaces.